arXiv:2305.00833v2 [cs.LG] 31 Oct 2023

# Learning to Reason and Memorize with Self-Notes

**Jack Lanchantin**[*]
Meta AI

**Shubham Toshniwal**[*]
NVIDIA

**Jason Weston**
Meta AI

**Arthur Szlam**
Meta AI

**Sainbayar Sukhbaatar**
Meta AI

## Abstract

Large language models have been shown to struggle with multi-step reasoning, and do not retain previous reasoning steps for future use. We propose a simple method for solving both of these problems by allowing the model to take *Self-Notes*. Unlike recent chain-of-thought or scratchpad approaches, the model can deviate from the input context at any time to explicitly think and write down its thoughts. This allows the model to perform reasoning on the fly as it reads the context and even integrate previous reasoning steps, thus enhancing its memory with useful information and enabling multi-step reasoning. Experiments across a wide variety of tasks demonstrate that our method can outperform chain-of-thought and scratchpad methods by taking Self-Notes that interleave the input text.

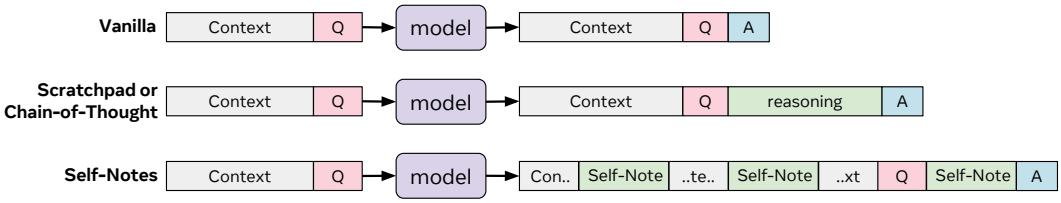


Figure 1: **[top] Vanilla** language models directly generate the answer (A) given the context and the question (Q). **[middle] Scratchpad and Chain-of-Thought** methods encourage the model to generate reasoning tokens before answering the question, but only after it has read the entire context. **[bottom] Self-Notes (ours)** allows the model to generate multiple internal reasoning notes that interleave the input context and question.

## 1 Introduction

Transformers [1] and similar variants have reshaped the field of machine learning with impressive results on sequence-based tasks [2]. Notably, large language models (LMs) such as GPT-3 [2] use transformers and are capable of solving various complex natural language tasks such as question answering. However, it's increasingly evident that there are still limitations to these models. Namely, transformers are limited in their ability to perform multi-step computations or store intermediate results due to the lack of an explicit internal dialogue or scratchpad [3, 4, 5].

When an LM is used for question answering (QA), it is typically fed a context prompt containing factual information along with a question, and then the model generates the answer directly, as shown in Fig. 1 (top). However, this autoregressive "one-step" approach struggles with multi-step reasoning

[*]Equal Contribution. Correspondence to `sainbar@meta.com`

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

tasks [6, 7, 8, 4]. We argue that this arises from the fact that vanilla LMs have a fixed computation budget for each token, and do not have the option to "think" more depending on the current context.

Recently, Chain-of-Thought and Scratchpad methods [3, 4, 9, 10] encourage the model to generate reasoning tokens or explain their answer one step at a time, leading to improved reasoning capabilities. These extra tokens are added before answering the question, but *after* it has read the full context and question, therefore postponing all "thoughts" until the end, as illustrated in Fig. 1 (middle).

In addition to the "one-step" problem, transformers lack a recurrent memory for state-tracking and solving highly nonlinear tasks [11], something that recurrent predecessor models such as the LSTM [12] are well equipped for. Modifications to the feed-forward transformer architecture that use a recurrent mechanism improve state-tracking results [11, 13, 14], but still use a fixed computation amount for a given prompt.

In this paper, we propose an approach that simultaneously makes the challenges in multi-step reasoning and state-tracking memory more tractable. Our method, "*Self-Notes*", allows the LM to deviate from the context prompts at any time to generate explicit reasoning tokens. Unlike a scratchpad or chain-of-thought, the model can interleave generated tokens with the input context and question, as demonstrated in Fig. 1 (bottom). Such Self-Notes can act as both explicit intermediate reasoning steps and working memory for state-tracking. Overall, we see the ability of a model to think as it reads, storing those thoughts for later use, as an important aspect of an intelligent agent, similar to how humans read [15]. Consider answering a question after reading the contents of a book. One would not want to re-read and re-reason over the whole book again to answer a second question. This issue is not addressed by either vanilla language models or chain-of-thought approaches.

To give an example, when reading a text, at any time there may be a reasoning step that requires combining two facts. The resulting inference can be written into a Self-Note and used for future reasoning, thus acting as an intermediate reasoning step. For example, given "`Alice has the box`" and "`Alice is at the park`" one can infer "`The box is at the park`" and write it to a Self-Note, which can be further combined with a future statement "`The key is in the box`" to conclude that "`The key is at the park`". Additionally, the Self-Note can act as a form of working memory because the model can write the latest state of an entity as new tokens while it reads the context. For example, in a programming environment, assume `x=5` initially, and then `x` gets incremented by `1`, the model can write the new state `x=6` and add it to its working memory.

We test our method on seven text datasets designed to evaluate multi-step reasoning and state-tracking: a proposed synthetic Toy-Story task, two synthetic program evaluation tasks [11, 16], two real-world chess game tasks [17], and two math word problem tasks previously used to test chain-of-thought prompting, MultiArith and GSM8K [18, 19]. Across these tasks, we consider four different paradigms of learning Self-Notes: supervised, semi-supervised, unsupervised, and few-shot prompted. Across all tasks and learning paradigms, our method outperforms both an LM which does not do any explicit reasoning, as well as chain-of-thought or scratchpad baselines.

## 2 Method

Let us consider an autoregressive transformer model $\mathcal{M}$ that predicts the next token in a sequence,

$$x_{t+1} = \mathcal{M}(x_1, ..., x_t).$$

Such a model, $\mathcal{M}$ is the foundation of many tasks like language modeling and question answering. In such tasks, the model is given a context $C = \{x_1, ..., x_t\}$ and potentially a question $Q$ as input and asked to generate $A$ which is the sequence of next words or an answer to a question.

Our Self-Notes method expands the capability of $\mathcal{M}$ by allowing it to enrich context $C$ with "note tokens" $n_i$ before producing the final output $A$. Note tokens share the same vocabulary as input tokens, but they are generated by the model itself. Self-Notes generated in this way can interleave with the context tokens and therefore can be used for writing down a newly inferred fact or tracking variable values.

At inference time, while processing input tokens $x_t \in C$ one by one, the model can start taking a note by generating a token that belongs to a predefined set of start tokens $N_{\text{sta}}$. In other words, at any point in the input text, if the next most probable token is in $N_{\text{sta}}$, then the model can autoregressively generate itself a note. A note ends when the model generates an end token $n_i \in N_{\text{end}}$, or after a

Table 1: Input-Output pairs for the Vanilla, Scratchpad, and Self-Notes method on Toy-Story and Algorithmic tasks. The input consists of the `input context` and the `question`, the `answer` is to be generated by the model. The highlighted reasoning steps are not given at test time and is to be generated by the model.

| Task | Vanilla | Scratchpad | Self-Notes |
|---|---|---|---|
| Toy-Story | Mary has the ball.<br>The ball is inside the box.<br>The key is inside the box.<br>Q: Who has the key?<br>Mary has the key. | Mary has the ball.<br>The ball is inside the box.<br>The key is inside the box.<br>Q: Who has the key?<br>[Q: Who has the box?<br>Mary has the box.<br>Q: Who has the key?<br>Mary has the key.]<br>Mary has the key. | Mary has the ball.<br>The ball is inside the box.<br>SQ: Who has the box?<br>Mary has the box.<br>The key is inside the box.<br>SQ: Who has the key?<br>Mary has the key.<br>Q: Who has the key?<br>Mary has the key. |
| Algorithmic | e = 3 ; e ++ ;<br>i = 3 ; if i < e : e ++ ;<br>print e e = 5 ; | e = 3 ; e ++ ;<br>i = 3 ; if i < e : e ++ ;<br>print e<br>[e = 3 ; e ++ ;<br>print e e = 4 ;<br>i = 3 ; if i < e : e ++ ;<br>print e e = 5 ;] e = 5 ; | e = 3 ; e ++ ;<br>print e e = 4;<br>i = 3 ; if i < e : e ++ ;<br>print e e = 5;<br>print e e = 5 ; |

fixed number of tokens are generated. Once the note ends, the generated note tokens are appended to the context where the start token was generated, and the model continues to process the rest of the input tokens. For example, a context $C = \{x_1, x_2, x_3, x_4\}$ can be enriched to become $\{x_1, x_2, n_1, n_2, n_3, x_3, x_4\}$ if the start token is generated after $x_2$:

$$n_1 = \mathcal{M}(x_1, x_2) \in N_{\text{sta}}, \quad n_2 = \mathcal{M}(x_1, x_2, n_1) \notin N_{\text{end}}, \quad n_3 = \mathcal{M}(x_1, x_2, n_1, n_2, ) \in N_{\text{end}}.$$

By repeating this mechanism, the context $C$ can be enriched with multiple notes at different locations. An overview of our method is shown in Figure 1 (bottom). A detailed example of how the model takes Self-Notes at inference is shown in Figure 5 (bottom).

The model can use notes as a form of working memory by writing information that might be useful in the future. It can also use a note as an intermediate reasoning step by inferring new facts as it reads. In particular, it can ask a question and answer it within it. This is useful in multi-step reasoning where a final question requires answering multiple sub-questions. Unlike implicit reasoning occurring internally within $\mathcal{M}$, Self-Notes are fed back to the model, making it available to future reasoning steps. This feedback loop also allows the model to overcome the limitation of transformers as a feedforward network [11], making it possible to do state-tracking. We explain four different paradigms to encourage the model to write Self-Notes as follows.

**Supervised Self-Notes.** One way to train $\mathcal{M}$ to generate useful notes is to use supervised learning on data that is enriched with "ground-truth" Self-Notes interspaced within the context. This training procedure is simple as we just have to train $\mathcal{M}$ on this enriched data using the standard LM training loss. After training, we can use $\mathcal{M}$ to generate Self-Notes, so we can apply it to test data that does not contain any Self-Notes or reasoning labels. $\mathcal{M}$ can generate a Self-Note at test time by predicting the next token in the context to be from $N_{\text{sta}}$.

**Semi-supervised Self-Notes.** We also consider a semi-supervised setting where only a subset of the training samples have ground truth Self-Notes. In this case, we prepend a special token $s$ to training samples without Self-Notes and train all samples with the standard LM loss: $C = \{s, x_1, ..., x_t\}$. As a result, the model is conditioned to generate Self-Notes during test time because the test context does not contain the special token $s$ prefix. This signals to the model that it should do extra reasoning and generate Self-Notes.

**Unsupervised Self-Notes.** We introduce a method for utilizing Self-Notes when no ground truth note is available for training.[2] This method relies on the fact that when the model is trained using the LM loss on all tokens in a QA task, it learns to not only generate answers but also questions. We leverage this property by letting the model generate its own questions and insert their answers as Self-Notes (i.e., interleaved throughout the context) during test time.

[2]We call this method "unsupervised" or "unlabeled" Self-Notes because there is no supervision or labeling on the notes, but there is full supervision on the output answers.